

Part 3, 4: Real-Time Booking Processing And UI (Last Part)

1. Overview

This part handles the actual booking action: a user chooses a seat and the system must process the request while preventing two users from successfully booking the same seat at the same time.

2. Objectives

1. Build a booking service (API) that allows users to book and cancel seats.
2. Ensure that concurrent requests for the same seat are handled correctly.
3. Update seat availability after every successful booking or cancellation.
4. Store booking changes so that they can be used by the rest of the system.

3. Requirements

1. **Book Seat** function that takes a user, event, and seat, and returns success or failure.
2. **Cancel Booking** function.
3. Prevent double-booking when **multiple users request the same seat concurrently**.
4. Reject bookings when the seat is already booked or when there are no available seats.
5. Keep the current seat status accessible to the rest of the system.
6. Store successful bookings and cancellations in the distributed storage from Part 1.
7. Provide a test that sends **multiple concurrent booking requests** for the same seat and demonstrates that only one request succeeds.
8. Provide a test for concurrent booking requests for **different seats**, demonstrating that valid requests can be processed successfully.
9. The booking service should return a clear result for each request, such as:
 - Booking successful.
 - Seat already booked.
 - Seat unavailable.
 - Booking cancelled.
10. The implementation must maintain a consistent seat status after concurrent requests.

4. UI (Local Host Website)

1. Provide a simple local host website (UI) that interacts with the booking service.
2. Display current seat availability for an event on the website.
3. Allow a user to book or cancel a seat from the website, calling the Book Seat and Cancel Booking functions.

4. Show the booking result on the website (booking successful, seat already booked, seat unavailable, booking cancelled).
5. Update the seat status shown on the website after each booking or cancellation.

5. Deliverables

1. Booking service source code containing book/cancel functions or API.
2. **A test showing multiple users attempting to book the same seat concurrently, where only one succeeds.**
3. A test showing multiple users booking different seats successfully.
4. Local host website (UI) source code allowing users to view seat availability and book/cancel seats.